PROJECT REPORT (CSN372)

A report submitted in partial fulfilment of the requirement for the course

Part of the degree of

BACHELOR OF TECHNOLOGY in

COMPUTER SCIENCE AND ENGINEERING

Submitted to

Dr. Sunil Ghildiyal

Assistant Professor

Submitted by

Name- Nihit Singh

Roll No: 23A03D0313

SAP ID: 1000020135

SCHOOL OF COMPUTING

DIT UNIVERSITY, DEHRADUN

(State Private University through State Legislature Act No. 10 of 2013 of Uttarakhand and approved by UGC)

Mussoorie Diversion Road, Dehradun, Uttarakhand - 248009, India.

April, 2026

Collaborative Whiteboard

1. Introduction

The increasing demand for collaborative tools in education and professional environments has led to the development of real-time applications that allow multiple users to interact simultaneously. Traditional applications often lack real-time synchronization, making collaboration inefficient and slow.

This project, a **Real-Time Collaborative Whiteboard**, provides a simple and efficient solution for multiple users to draw and interact on a shared canvas in real time. The application uses **WebSockets (Socket.io)** to enable instant communication between users without the need for page refresh.

The frontend is developed using **React.js**, while the backend uses **Node.js with Express and Socket.io** for handling real-time events. The system ensures that all connected users can see updates instantly, making collaboration seamless and interactive.

This project demonstrates how modern web technologies can be used to build lightweight and responsive real-time collaborative systems.

2. Problem Definition

Traditional web applications follow a request-response model, which is not suitable for real-time collaboration. Users often need to refresh the page or wait for updates, resulting in delays and poor user experience.

Existing tools like Google Docs or online whiteboards provide real-time features but are often complex and not customizable for specific use cases. There is a need for a simple and lightweight system that allows:

- Real-time collaboration without delays
- Multiple users interacting on the same platform
- Easy access without complex authentication
- A clean and intuitive user interface

This project addresses these challenges by implementing a WebSocket-based system that ensures instant synchronization between all users.

3. Existing System

Current solutions for collaboration include:

- **Google Docs / Whiteboards** – Feature-rich but complex and heavy
- **Messaging Apps** – Provide real-time communication but lack drawing/canvas features
- **Traditional Web Apps** – Use HTTP polling, which causes delays

These systems either lack customization, are too complex, or do not provide efficient real-time drawing capabilities.

4. Proposed System

The proposed system is a **Real-Time Collaborative Whiteboard** that allows multiple users to draw simultaneously on a shared canvas.

Architecture:

- **Frontend:** React.js (Canvas API for drawing)
- **Backend:** Node.js + Express
- **Real-time Communication:** Socket.io (WebSockets)

Key Features:

- Real-time drawing synchronization
- Multiple users in a single room
- Unique username-based room access
- Role system (Creator, Editor, Viewer)
- Cursor tracking with user names
- Clear board functionality

The system uses an event-based communication model where drawing actions are broadcast to all connected users instantly.

5. System Requirements

1. Hardware Requirements:
    - Processor: Intel i3 or above
    - RAM: 2 GB minimum (4 GB recommended)
    - Storage: 500 MB free space
    - Network: Stable internet connection for WebSocket communication

2. Software Requirements:
    - Backend: Node.js, Express.js
    - Frontend: React.js(Vite), HTML5 Canvas
    - Real-time Communication: Socket.io
    - Database: MySQL
    - Development Tools: IDE (IntelliJ IDEA, Eclipse, or VS Code)
    - Browser: Google Chrome, Microsoft Edge, or any modern web browser

6. Modules of System

1. **User Module**
    - Handles user entry through username-based login
    - Assigns a unique identifier (UID) to each user
    - Prevents duplicate usernames within the same room
    - Stores user details using local storage
2. **Room Management Module**
    - Allows users to create or join rooms using a room code
    - Maintains list of active participants in a room
    - Handles user join and leave events
    - Ensures isolated collaboration within each room
3. **Drawing Module**
    - Provides canvas-based drawing functionality
    - Supports tools like pen and eraser
    - Captures coordinates, color, and brush size
    - Renders drawings in real time

4. **Real-Time Communication Module**
    - Uses Socket.io for real-time communication
    - Synchronizes drawing actions across users
    - Enables instant updates without page refresh
    - Ensures smooth collaboration
5. **Participant Management Module**
    - Displays list of users in the room
    - Manages roles: Creator, Editor, Viewer
    - Allows creator to assign permissions
    - Controls user access to drawing features
6. **Cursor Tracking Module**
    - Tracks real-time cursor movement of users
    - Displays cursor with username label
    - Helps users understand others' interactions
    - Removes cursor when it leaves canvas area

7. Implementation

The system is implemented using a client-server architecture where the frontend handles user interaction and the backend manages real-time communication using WebSockets.

1. Canvas Coordinate Mapping

To ensure accurate drawing, the application maps screen coordinates to the internal canvas resolution. This avoids offset issues when the canvas is resized.

```
const getCoords = (e) => {
  const rect = canvasRef.current.getBoundingClientRect();
  const scaleX = canvasRef.current.width / rect.width;
  const scaleY = canvasRef.current.height / rect.height;

  return {
    x: (e.clientX - rect.left) * scaleX,
    y: (e.clientY - rect.top) * scaleY,
  };
};
```

This approach ensures consistent drawing regardless of screen size or zoom level.

2. Drawing and Eraser Implementation
The application supports both pen and eraser tools using canvas compositing techniques.

```
const draw = (e) => {
  if (!drawing) return;
  const { x, y } = getCoords(e);
  const ctx = canvasRef.current.getContext("2d");

  ctx.globalCompositeOperation =
    tool === "eraser" ? "destination-out" : "source-over";
  ctx.strokeStyle = color;
  ctx.lineWidth = tool === "eraser" ? size * 3 : size;
  ctx.lineTo(x, y);
  ctx.stroke();

  socket.emit("draw", { roomCode, x, y, color, size, tool });
};
```

The eraser removes pixels instead of drawing white strokes, ensuring correct behavior.

3. Real-Time Communication using Socket.io
All drawing actions are transmitted to other users in real time.

```
socket.on("draw", (data) => {
  ctx.lineTo(data.x, data.y);
  ctx.stroke();
});
```

This ensures that all users see the same drawing instantly.

4. Cursor Tracking
The system tracks user cursor positions and displays them with usernames.

```
const handleCursor = (e) => {
  const rect = canvasRef.current.getBoundingClientRect();
  const inside =
    e.clientX >= rect.left &&
    e.clientX <= rect.right &&
    e.clientY >= rect.top &&
    e.clientY <= rect.bottom;

  if (!inside) {
    socket.emit("cursor-leave", { roomCode });
    return;
  }
  socket.emit("cursor-move", {
    roomCode,
    x: e.clientX,
    y: e.clientY,
    name: user.name,
  });
};
```

This helps users visualize others' interactions on the board.

5. Board State Synchronization

When a new user joins, previous drawing data is loaded and rendered.

```
socket.on("load-board", (actions) => {
  ctx.clearRect(0, 0, canvas.width, canvas.height);

  actions.forEach((a) => {
    ctx.globalCompositeOperation =
      a.tool === "eraser" ? "destination-out" : "source-over";

    if (a.type === "start") {
      ctx.beginPath();
      ctx.moveTo(a.x, a.y);
    } else {
      ctx.lineTo(a.x, a.y);
      ctx.stroke();
    }
  });
});
```

This ensures that new users see the complete board instead of a blank canvas.

6. Backend Room and Role Management

The backend manages users, roles, and room state efficiently.

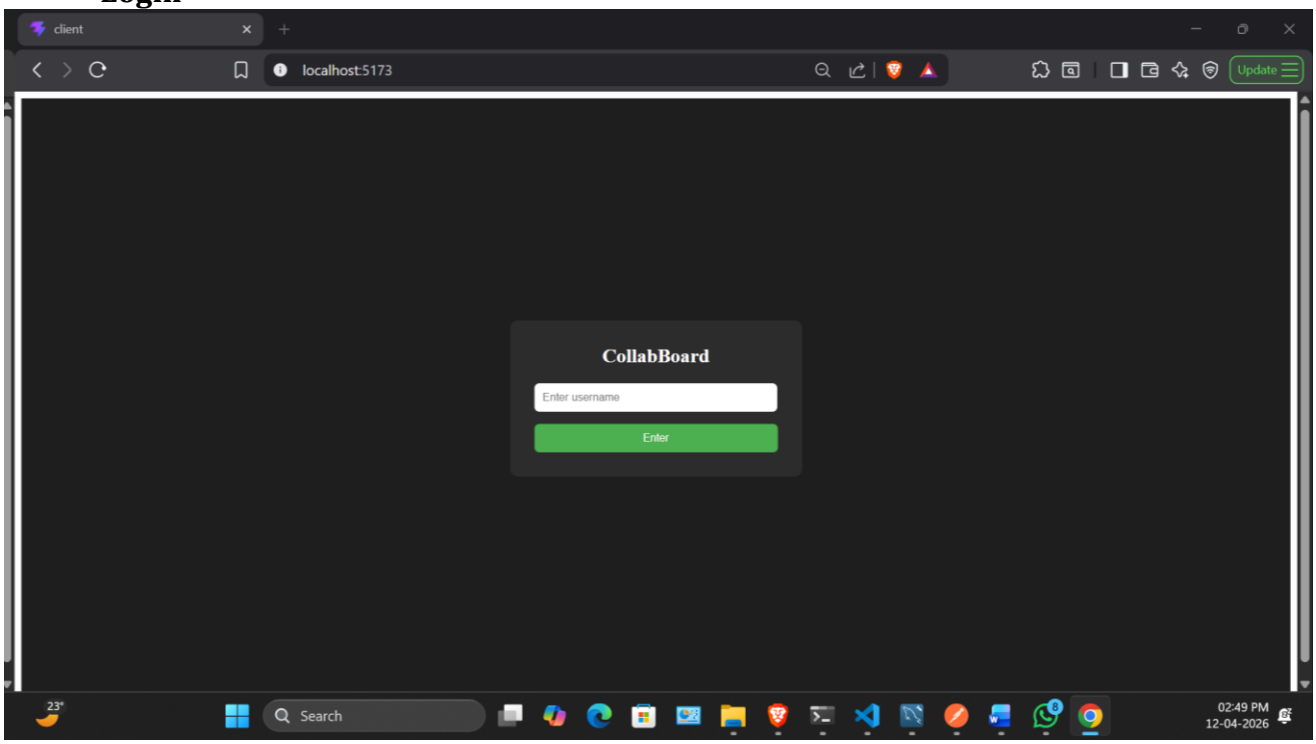
```
socket.on("join-room", ({ roomCode, name, uid }) => {
  socket.join(roomCode);

  roomUsers[roomCode] = roomUsers[roomCode].filter(
    (u) => u.uid !== uid
  );

  let role =
    userRoles[roomCode]?.[uid] ||
    (roomUsers[roomCode].length === 0 ? "creator" : "viewer");

  userRoles[roomCode] = {
    ...userRoles[roomCode],
    [uid]: role,
  };

  const user = { uid, socketId: socket.id, name, role };
  roomUsers[roomCode].push(user);

  socket.emit("load-board", roomBoards[roomCode] || []);
  io.to(roomCode).emit("participants", roomUsers[roomCode]);
});
```

This ensures:

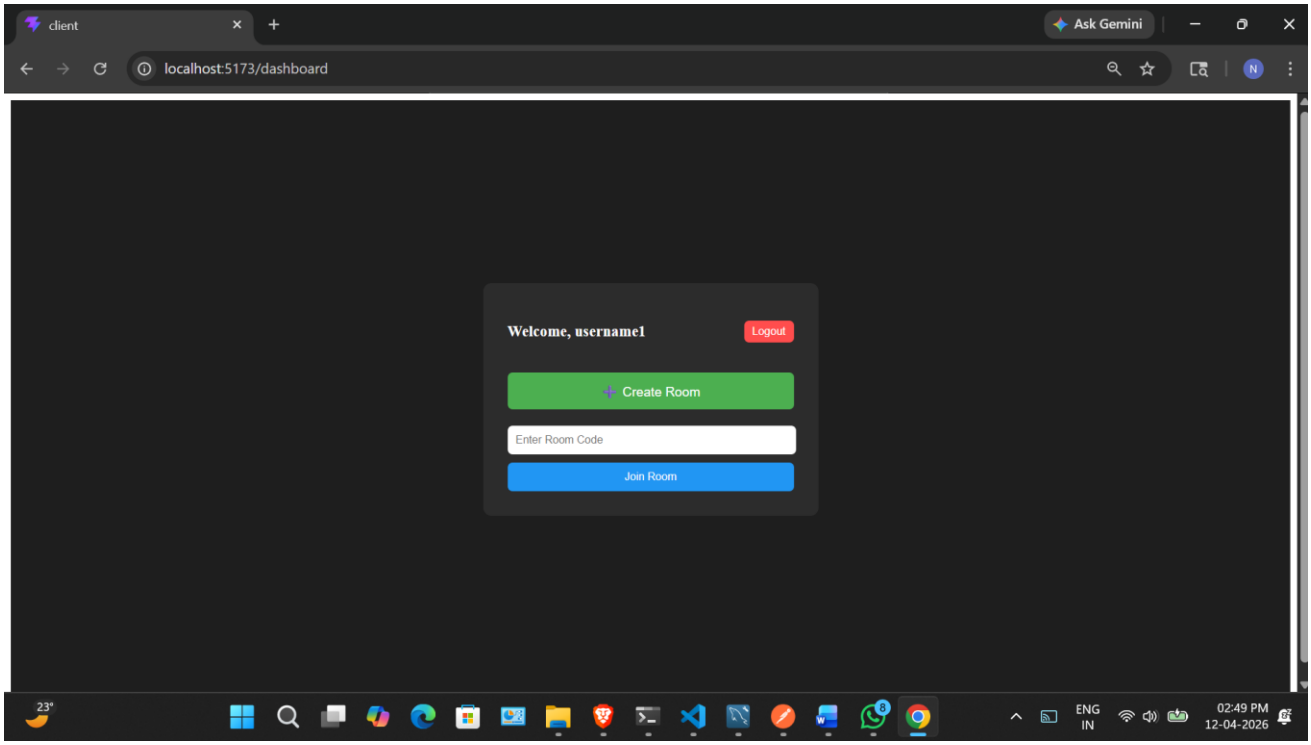
- No duplicate users on refresh
- Role persistence
- Proper synchronization

8. Project Screenshot

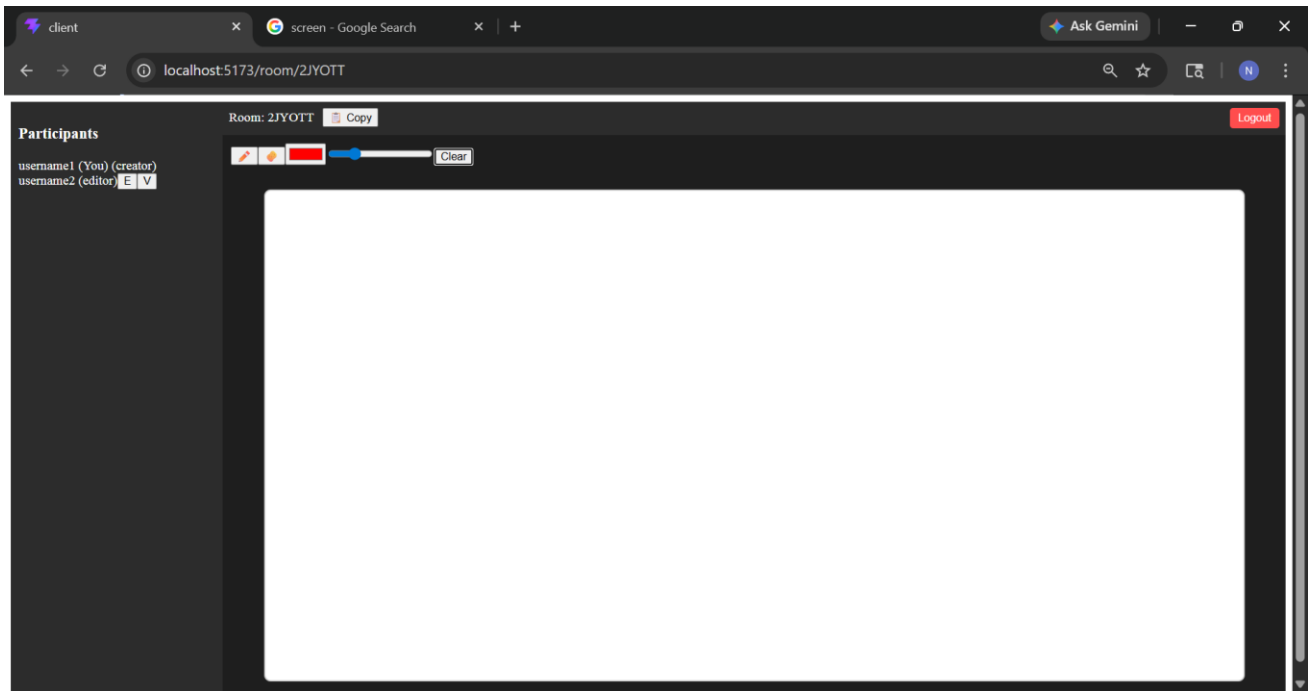
- **Login**



- **Dashboard**



- **Whiteboard Room**
    - UI of a room creator



    - (2 screens/users connected in a room)